

- // Proceedings of the USENIX Summer 1994 Technical Conference on USENIX Summer 1994 Technical Conference. 1994, 1: 6.
- [3] LAMETER C. Slub: the unqueued slab allocator v6 [EB/OL]. [2022-12-06]. <https://lwn.net/Articles/229096/>.
- [4] MACKALL M. slob: introduce the slob allocator [EB/OL]. [2022-12-06]. <https://lwn.net/Articles/157944/>.
- [5] DENNING P J. The working set model for program behavior [J]. Commun, 1968, 11 (5): 323-333.
- [6] BOSMAN E, RAZAVI K, BOS H, et al. Dedup est machina: memory deduplication as an advanced exploitation vector [C] // 2016 IEEE Symposium on Security and Privacy (SP), .2016: 987-1004.
- [7] ARM. Programmer's guide for armv8-a [EB/OL]. [2022-12-06]. https://static.docs.arm.com/den0024/a/DEN0024A_v8_architecture_PG.pdf.
- [8] Arm cortex-a57 mpcore processor [EB/OL]. [2022-12-06]. http://infocenter.arm.com/help/topic/com.arm.doc.ddi0488c/DDI0488C_cortex_a57_mpcore_r1p0_trm.pdf.
- [9] Cache coloring [EB/OL]. [2020-05-01]. https://en.wikipedia.org/wiki/Cache_coloring.
- [10] Introduction to cache allocation technology in the intel® xeon® processor e5 v4 family [EB/OL]. [2020-05-01]. <https://software.intel.com/en-us/articles/introduction-to-cache-allocation-technology>.
- [11] Memory system resource partitioning and monitoring (mpam), for armv8-a [EB/OL]. [2020-05-01]. https://static.docs.arm.com/ddi0598/a/DDI0598_MPAM_supp_armv8a.pdf.
- [12] LEPERS B. Improving performance on NUMA systems [D]. PhD thesis, 2014.
- [13] ZHANG K, CHEN R, CHEN H. Numa-aware graphstructured analytics [C] // Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. 2015: 183-193.

物理内存管理：扫码反馈



进程与线程

除了使用虚拟内存对物理内存资源进行抽象以外，操作系统也对处理器资源提供了抽象，即进程（Process）与线程（Thread）。进程就是运行中的程序，它为程序提供了“独享”的处理器资源，从而简化了程序的编写。而针对进程间数据不易共享、通信开销高等问题，操作系统又在进程内部引入了更加轻量级的执行单元，也就是线程。在前面的章节中，我们已经从程序员视角接触了进程与线程的概念及其对应的操作接口，而本章将从操作系统视角出发，介绍操作系统对进程/线程的内部表示、相关接口的实现方法以及进程/线程切换的细节等内容。

6.1 进程的内部表示与管理接口

本节主要知识点

- 进程在内核中是如何表示的？
- 进程的基本管理接口是如何实现的？
- 进程的执行状态是什么，有什么作用？

6.1.1 进程的内部表示——PCB

3.3.1 节已经介绍过，为了使多个程序“轮流”使用处理器（分时复用），操作系统为每个程序对应的进程维护了结构体 `process`，保存了进程包含的处理器上下文信息。而在第 4 章引入虚拟内存之后，`process` 结构体被进一步扩展，包含虚拟地址空间相关的内容。由于该结构体包含与进程相关的关键信息，我们也将其称为进程控制块（Process Control Block, PCB）。

为了支撑进程在用户态和内核态的运行，只维护上下文和虚拟内存是不够的。尽管进程在初始化时已经分配了用于在用户态执行的栈结构，但当它切换至内核态后，操作

系统也需要栈结构来存储临时变量，且栈内保存的内容不能被进程随意访问和修改。因此，操作系统为每个进程都提供了内核栈，与进程在用户态使用的用户栈进行区分。当进程切换到内核执行时，其栈寄存器地址就会从用户栈切换到内核栈。代码片段 6.1 因而对 `process` 结构体（见 4.2.4 节）做了进一步扩展，包含处理器上下文（`ctx`）、虚拟地址空间（`vmSPACE`）和内核栈（`stack`）。该结构是本章 PCB 数据结构的第一版，之后将以该结构作为例子并继续扩展，讲解进程及其相关管理接口的实现。

代码片段 6.1 PCB 结构（第一版）：在进程结构体基础上加入内核栈

```

1 // 一种简单的 PCB 结构实现
2 struct process_v1 {
3     // 处理器上下文
4     struct context *ctx;
5     // 虚拟地址空间（包含页表基地址）
6     struct vmSPACE *vmSPACE;
7     // 内核栈
8     void *stack;
9 };

```

6.1.2 进程创建的实现

第 3 章已经以简单的 `shell` 程序为例介绍了进程创建、退出、等待等接口（系统调用）的用法。那么，当这些接口被调用时，究竟在操作系统内部会发生什么呢？本节之后的内容将继续以 `shell` 作为线索，介绍操作系统是如何管理进程的。

从用户视角来看，进程创建只需要简单的一个系统调用或者一条命令（如在 `shell` 中输入的一条 `ls` 命令）即可完成。不过，从实现的角度看，进程的创建要复杂得多，涉及 PCB 创建、虚拟内存初始化、可执行文件加载等多个步骤。代码片段 6.2 展示了进程创建的伪代码实现，下面将详细介绍其具体步骤。

代码片段 6.2 进程创建的伪代码实现

```

1 int process_create(char *path, char *argv[], char *envp[])
2 {
3     // 创建一个新的 PCB，用于管理新进程
4     struct process *new_proc = alloc_process();
5     // 虚拟内存初始化：初始化虚拟地址空间及页表基地址
6     init_vmSPACE(new_proc->vmSPACE);
7     new_proc->vmSPACE->pgtbl = alloc_page();
8
9     // 内核栈初始化
10    init_kern_stack(new_proc->stack);
11
12    // 加载可执行文件并映射到虚拟地址空间
13    struct file *file = load_elf_file(path);
14    for (struct seg loadable_seg : file->segs)
15        vmSPACE_map(new_proc->vmSPACE, loadable_seg);

```

```
16
17 // 准备运行环境：创建并映射用户栈
18 void *stack = alloc_stack(STACKSIZE);
19 vmSPACE_map(new_proc->vmSPACE, stack);
20
21 // 准备运行环境：将参数和环境变量放到栈上
22 prepare_env(stack, argv, envp);
23 // 上下文初始化
24 init_process_ctx(new_proc->ctx);
25 // 返回
26 }
```

1. 创建进程控制块。当 process_create 被调用时，内核会首先创建一个新的 PCB，用于维护子进程的相关信息。

2. 虚拟内存初始化。独立的虚拟地址空间是进程必不可少的组成部分。因此，内核在创建 PCB 后将为进程创建单独的页表，即申请一个物理页作为顶级页表页，该页的起始地址即为页表基地址。在进程的创建过程中，内核会在该顶级页表页的基础上不断地添加更多映射。

3. 内核栈初始化。为了之后在内核中处理与进程相关的逻辑（如系统调用），内核还会预先分配物理页，作为进程的内核栈。

4. 加载可执行文件到内存。在完成 PCB 的初始化后，内核将以可执行文件形式保存在硬盘中的程序加载到进程的虚拟内存空间中。为了方便操作系统加载，可执行文件通常都有固定的格式。例如，Linux 中常用的可执行文件格式为可执行和可链接格式（Executable and Linkable Format, ELF），其结构如图 6.1 所示。除了保存程序运行时所需的代码和数据等信息以外，可执行文件中还保存了一项重要的内容——程序的起始地址。该地址在加载过程中会被内核记录下来，并在初始化处理器上下文时使用。

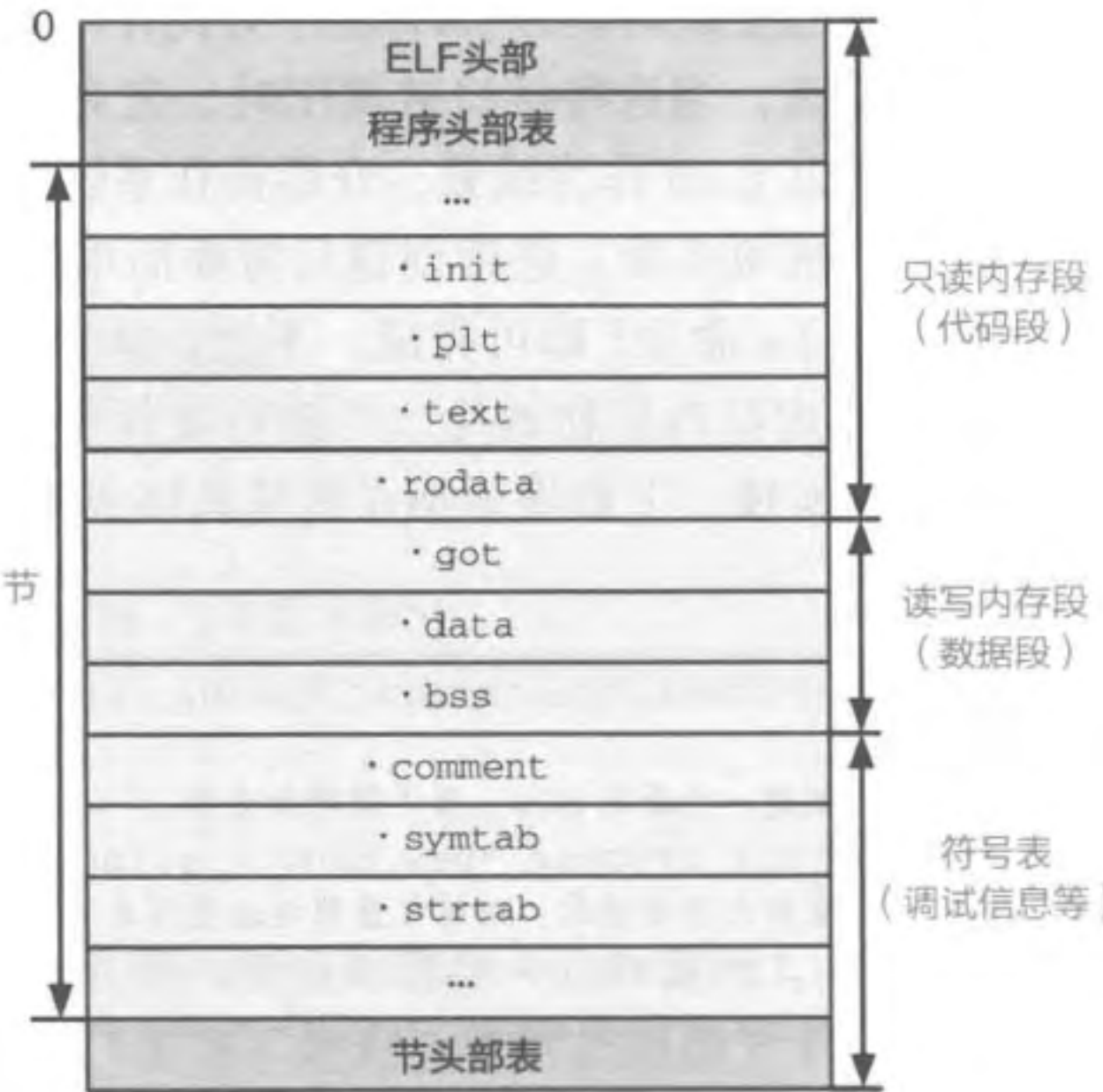


图 6.1 ELF 结构

小知识：ELF

ELF 文件的起始部分是 ELF 头部（ELF header）和程序头部表（Program Header

Table), 之后跟着若干连续的程序节 (Program Section), 最后是节头部表 (Section Header Table)。

ELF 文件里包含的内容较多, 但并不是整个文件都需要被加载到内存。ELF 文件在节头部表中已经标注了所有需要加载的程序节, 内核会将这些程序节加载到进程的虚拟地址空间。一般来说, 需要加载的程序节包括:

- `.init`: 程序的初始化代码。
- `.text`: 程序代码, 是由一条条的机器指令组成的。
- `.rodata`: 程序中的只读数据, 包括一些不可修改的常量数据, 例如全局常量、`char *str = "apple"` 中的字符串常量等。然而, 如果使用 `char str2[] = "apple"`, 那么此时该字符串是动态存在栈上的。
- `.data`: 程序中初始化的全局变量或 C 语言的静态变量数据。定义在函数内部的局部非静态变量不在该段中存储。
- `.bss`: 程序中未初始化的全局变量或 C 语言的静态变量, 例如 `int a`。由于在运行期间未初始化的全局变量被初始化为 0, 因此链接器只在符号表中记录, 而不占用实际空间。

为了方便管理, ELF 文件使多个连续且具有相似特征的程序节组成段 (Segment), 并以段为单位进行加载。如图 6.1 所示, ELF 文件有两个主要的段: 只读内存段 (代码段) 和读写内存段 (数据段)。代码段中会保存代码 (`.init`、`.text`) 和只读数据 (`.rodata`), 这一部分的内容在拷贝到内存后的权限是可读可执行^①。而数据段中存放着源文件中定义的全局变量和静态变量 (初始化数据节 `.data` 和未初始化数据节 `.bss`), 这一段加载到内存后的权限是可读可写。在加载过程中, 内核会以段为单位映射程序节 (代码片段 6.2), 并在页表上设置相应的权限。

5. 初始化用户栈及运行环境。在完成可执行文件的加载后, 内核开始为进程准备运行所必要的环境, 使处理器返回用户态后可以顺利运行。进程在运行时, 代码、数据和栈结构都是必不可少的, 前两种已经在可执行文件加载后完成了初始化, 但用户态执行所需的栈结构 (即用户栈) 还没有准备好。因此, 内核会申请物理内存并创建用户栈, 然后通过填写页表项将其映射到进程的虚拟地址空间中。至于第 3 章介绍的进程虚拟地址空间布局中包含的其他部分 (如堆和共享代码库), 并不是所有程序都需要, 因此可以在进程执行过程中按需进行初始化。

在完成栈结构的映射后, 内核还需要准备应用程序运行时所需的环境。以 C 语言编写的程序为例, POSIX 规定其 `main` 函数通常包含三个参数: 参数数量 `argc`、运行所

① 将只读数据和代码合并为一个段其实是一种优化, 可以减少虚拟内存映射相关的函数调用次数。通过为链接器配置 `-rosegment` 参数, 我们也可以将只读数据映射为只读不可执行。

需参数 `argv` 和环境变量（包含用于描述运行时环境的变量，例如动态链接库的文件路径）`envp`。因此，内核会将这些变量放在用户栈上，并得到如图 6.2 所示的栈结构。

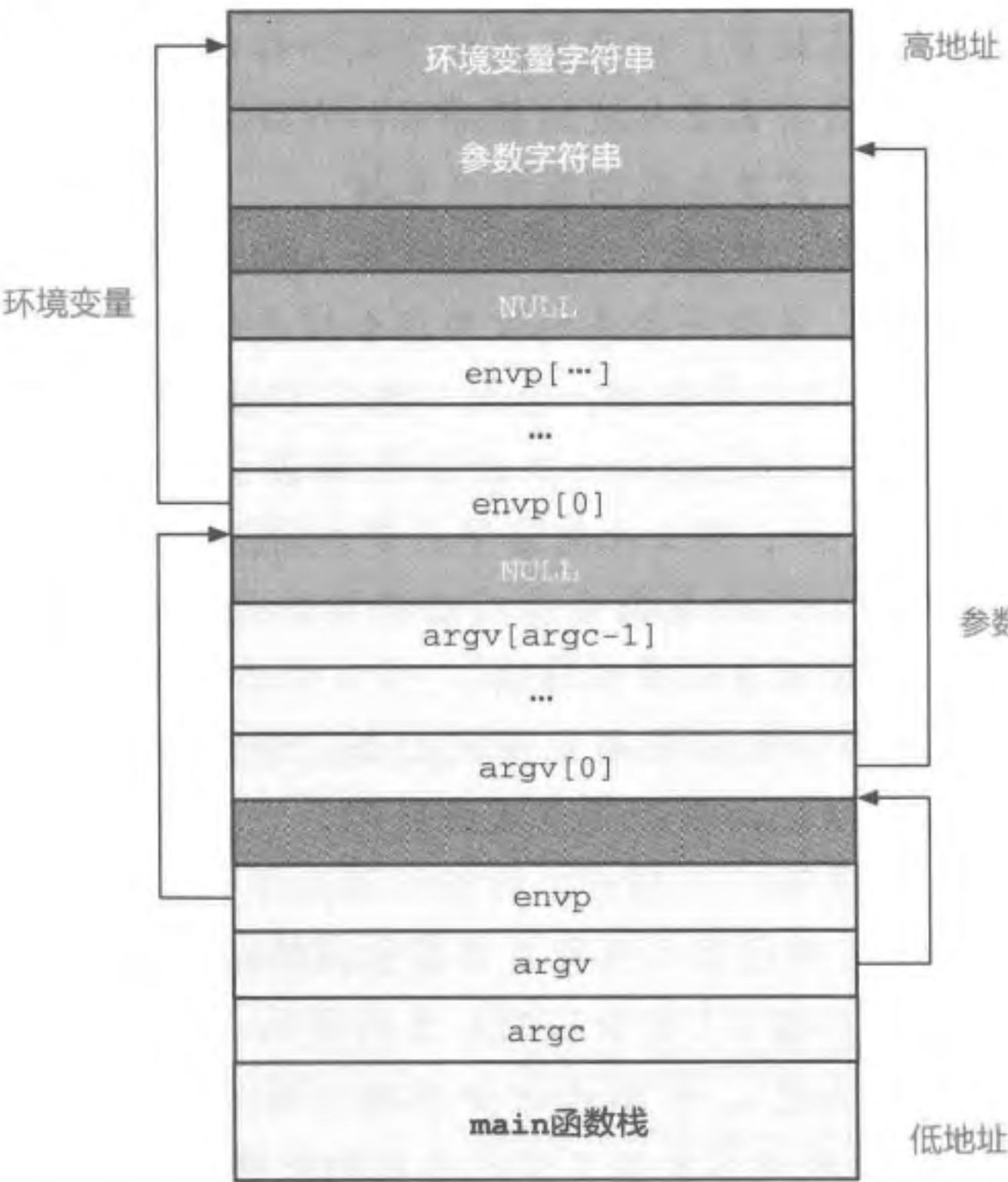


图 6.2 为进程创建的运行时栈：以 C 语言程序为例

6. 处理器上下文初始化。接下来，内核还需要准备好进程的处理器上下文，使其能开始执行。由于新进程还没有执行过任何代码，因此大部分寄存器都未使用过，可以直接赋值为 0。但是，由于特殊寄存器（PC、SP 和 PSTATE）保存了与硬件状态相关的信息，需要专门考虑。

- PC。在加载可执行文件时，内核已经获取了程序的起始地址，应当把该地址赋值给 PC 寄存器。可是，此时 PC 寄存器正在被内核使用，不能直接赋值。第 3 章介绍过，在内核态切换到用户态时，硬件会自动将 ELR_EL1 寄存器中的值恢复到 PC 寄存器中。因此，此时将程序的起始地址放入 ELR_EL1 寄存器即可。
- PSTATE。进程开始执行之前，操作系统会为该寄存器设置一个初始值（如 0xb0000）。与 PC 寄存器类似，此时不能直接设置 PSTATE，但可以通过修改 SPSR_EL1 寄存器，借由硬件机制在切换到用户态时设置 PSTATE。
- SP。在运行环境准备完成后，当前用户栈的栈顶地址应写入 SP 寄存器。由于 AArch64 处理器中的 SP 寄存器有多份，此时可以直接修改 SP_EL0，而不必担心影响内核执行（通常配置内核栈使用的是 SP_EL1）。

在操作系统完成上述6个步骤后，进程就已经准备好执行了。当处理器返回用户态时，PC寄存器被硬件自动设置为ELR_EL1的值，程序就会从对应的内存地址（即可执行文件中规定的起始地址）获取指令并开始执行。

小知识：从规定的起始地址到C程序入口函数

由于C程序会从main函数开始执行，读者可能会认为当进程返回用户态开始执行时，会直接从main函数的第一条指令开始执行。但实际上，程序的起始地址对应的是代码段中的_start函数，这是因为进程在执行main函数之前还需要进行一系列准备工作。因此，glibc等标准库会提供一套C运行时库（C Run Time Library）来完成这些功能。以glibc为例，其包含的crt1.o文件会提供_start的代码，而用户程序在编译的链接阶段会与glibc标准库进行链接，此时_start函数就合并到了ELF文件中，其所在地址也成为程序的起始地址。

那么，程序在执行到main函数之前，需要做哪些准备呢？首先，在main函数运行前，_start通过重置栈帧基地址寄存器（FP）和链接寄存器（LR）的值来构建初始栈帧，准备寄存器环境，以及读入内核为main函数准备的argc、argv和envp，然后调用__libc_start_main函数。之后，__libc_start_main负责注册一些句柄（如动态库释放资源的句柄rtld_fini、程序退出的句柄fini），然后调用ELF文件.init中的init函数。完成这一系列的工作后，__libc_start_main会调用main函数，真正进入程序的执行阶段。

6.1.3 进程退出的实现

进程通常不会无休止地运行下去。例如当shell中的ls进程打印出目录下的所有文件后，其应用逻辑已经执行完成，进程即将退出。由于内核在创建进程时也创建了PCB、页表等数据结构，因此在进程退出时需要对这些资源进行回收。所以，操作系统提供了用于进程退出的系统调用。

代码片段6.3展示了进程退出相关系统调用process_exit的伪代码实现。为了叙述简便，我们假设内核维护着curr_proc变量，并总是指向当前正在运行的进程的PCB（我们在之后的代码中也会一直使用该变量）。不难看出，process_exit的实现实际上就是将进程持有的资源（虚拟地址空间、处理器上下文、内核栈等）一一销毁和回收，最后销毁和回收PCB占据的内存资源。由于进程在调用process_exit之后将不会返回用户态继续执行，因此最后还需要调用schedule告知内核选择其他进程执行。schedule函数的功能很重要，因为它涉及内核的调度模块，即选择下一个需要执行的进程。我们将在6.1.8节介绍schedule的一种实现方法。

另外，process_exit的第一版实现中还存在一个问题：当调用destroy_kern_

stack 销毁内核栈时，该进程依然在使用自己的内核栈，此时可能会引起错误，该问题将在后续的版本中解决。

代码片段 6.3 进程退出的伪代码实现（第一版）

```
1 void process_waitpid_v1(int id)
2 {
3     while (TRUE) {
4         bool not_exist = TRUE;
5         // 扫描内核的进程列表，寻找对应进程
6         for (struct process *proc : all_processes) {
7             // 若发现该进程还在进程列表中，说明还未退出
8             if (proc->pid == id)
9                 not_exist = FALSE;
10        }
11        // 如果没有退出，则调度下个进程执行，否则直接返回
12        if (!not_exist)
13            schedule();
14        else
15            return;
16    }
17 }
```

小知识：C 程序对于进程退出系统调用的隐式调用

读者可能会好奇，很多 C 程序在退出前并不会调用 `process_exit`，它们只是使用 `return` 指令从 `main` 函数返回了。那么，这些程序对应的进程到底是怎么退出的呢？

实际上，这些进程是隐式地调用进程退出系统调用，而不是直接写在 C 程序中。例如在 `glibc` 中，由于 `main` 函数是由 `__libc_start_main` 函数调用的，因此当 `main` 函数返回时会返回该函数。此时，`__libc_start_main` 函数就会调用 `process_exit` 系统调用，从而使进程退出。

6.1.4 进程等待的实现

前面介绍的进程创建和退出都是针对单一进程的功能，并未考虑进程间的协作。而在实际应用中，进程之间常常是存在联系的。仍以 `shell` 场景为例，假设小明在 `shell` 中输入 `grep xiaoming` 搜索与自己名字相关的文件。如果小明的电脑里有很多文件，搜索过程会持续比较长的时间，此时 `shell` 会“卡住”，暂时不能接收新指令。当 `grep` 指令将所有文件名输出完毕后，`shell` 才恢复响应。在前面的学习中，我们已经知道 `shell` 进程可以通过 `process_create` 来创建 `grep` 进程。但是，`process_create` 函数在创建结束后就立即返回了，应该如何使 `shell` 进程保持“卡住”，直到 `grep` 进程退出才恢

复响应呢？为了满足这一需求，就需要引入进程等待相关的系统调用。

代码片段 6.4 展示了进程等待系统调用——`process_waitpid` 的伪代码实现。该系统调用将使调用进程在内核中等待，直到其监控的进程退出。为了使操作系统知道应该监控哪个进程，我们需要引入进程标识符（Process Identifier, `pid`），并将其作为 `process_waitpid` 的第一个参数。进程标识符存储在进程的 PCB 中（见代码片段 6.5），并作为进程创建时的返回值。`process_waitpid` 的实现逻辑比较简单，它遍历操作系统所有未退出的进程（假设操作系统用 `all_processes` 作为列表记录所有进程），寻找标识符与参数中的 `id` 匹配的进程。如果找不到匹配的进程，说明该进程已经退出，那么 `process_waitpid` 将立即返回，否则调用 `schedule`，让操作系统调度下一个需要执行的进程。如果操作系统恰好又选择让该调用进程执行，该进程会重复执行 `while` 循环中的逻辑，再次判断子进程是否退出。如果等待的进程已经退出，内核会将其从进程列表 `all_processes` 中删去，此时调用进程不再调用 `schedule`，而是返回用户态继续执行。

代码片段 6.4 进程等待的伪代码实现（第一版）

```

1 void process_waitpid_v1(int id)
2 {
3     while (TRUE) {
4         bool not_exit = FALSE;
5         // 扫描内核的进程列表，寻找对应进程
6         for (struct process *proc : all_processes) {
7             // 若发现该进程还在进程列表中，说明还未退出
8             if (proc->pid == id)
9                 not_exit = TRUE;
10        }
11        // 如果没有退出，则调度下个进程执行，否则直接返回
12        if (not_exit)
13            schedule();
14        else
15            return;
16    }
17 }
```

代码片段 6.5 PCB 结构（第二版）：加入进程标识符

```

1 struct process_v2 {
2     // 处理器上下文
3     struct context *ctx;
4     // 虚拟地址空间（包含页表基地址）
5     struct vmSPACE *vmSPACE;
6     // 内核栈
7     void *stack;
8     // 进程标识符
9     int pid;
10 }
```

```
10 };
```

通过 `process_waitpid` 的支持，操作系统已经能满足上述例子的需求。当 shell 进程调用 `process_create` 接口创建 `grep` 进程后，记录其返回值（`grep` 进程的标识符），并将其作为参数调用 `process_waitpid`。此后，shell 进程将在内核中等待，直到 `grep` 退出。当然，以上功能还依赖于内核对于进程列表 `all_processes` 的维护，例如在进程退出时将其对应 PCB 从列表中移除，本节在代码中省略了这部分细节。

6.1.5 `exit` 与 `waitpid` 之间的信息传递

目前 `process_waitpid` 的功能还比较受限：进程只知道被监控进程是否退出，并不知道它是正常退出还是异常退出。为了满足这一需求，我们可以使 `process_exit` 以返回值形式记录进程退出时的状态，而 `process_waitpid` 可以获取该状态。POSIX 接口也提供了类似的支持：其定义的进程退出系统调用 `exit` 接收一个整数类型的参数，可用于描述进程退出时的状态（如非 0 表示异常），而其他进程可以通过调用 `waitpid` 获取这一状态，并进行相应的处理。为了支持这一功能，操作系统需要对进程退出和等待的系统调用做进一步扩展。

代码片段 6.6 首先展示了 `process_exit` 系统调用的变化。为了表示进程退出状态，`process_exit` 引入了新的参数 `status`。在进程退出时，`status` 需要被保存到进程的 PCB 中，因此 PCB 也要进行扩展以支持保存退出状态 `exit_status`（代码片段 6.7）。由于 PCB 中保存了进程的退出状态，因此 `process_exit` 不能直接销毁 PCB，而要等到 `process_waitpid` 获取退出状态之后再行销毁。类似地，为了解决第一版中遗留的问题，`process_exit` 也不再销毁进程的内存栈。为了便于识别进程是否退出，PCB 中还引入了变量 `is_exit` 进行记录，在 `process_exit` 中会将其设为 `TRUE`。此时，操作系统就为进程引入了一个特殊的执行状态：它已经调用了 `process_exit`，不可能再返回用户态继续执行；但与此同时，由于它的 PCB 还未被销毁，因此仍作为一个进程存在，用户甚至能通过一些命令（如 shell 中的 `ps` 指令）观察到它。这种进程通常被称为僵尸（Zombie）进程。

代码片段 6.6 进程退出的伪代码实现（第二版）：在退出前维护退出状态 `exit_status` 和 `is_exit`

```
1 void process_exit_v2(int status)
2 {
3     // 销毁上下文结构
4     destroy_ctx(curr_proc->ctx);
5     // 销毁虚拟地址空间
6     destroy_vmspace(curr_proc->vmspace);
7     // 保存退出状态
8     curr_proc->exit_status = status;
9     // 标记进程为退出状态
10    curr_proc->is_exit = TRUE;
```



```

11 // 告知内核选择下个需要执行的进程
12 schedule();
13 }

```

代码片段 6.7 PCB 结构 (第三版): 加入进程退出相关状态 `exit_status` 和 `is_exit`

```

1 struct process_v3 {
2     // 处理器上下文
3     struct context *ctx;
4     // 虚拟地址空间 (包含页表基地址)
5     struct vmSPACE *vmSPACE;
6     // 内核栈
7     void *stack;
8     // 进程标识符
9     int pid;
10    // 退出状态
11    int exit_status;
12    // 标记是否退出
13    bool is_exit;
14 };

```

代码片段 6.8 展示了支持退出状态后 `process_waitpid` 的第二版实现, 其接收两个参数: 进程标识符 `id` 和用于传递进程退出状态的指针 `status`。在 `process_waitpid` 的执行过程中, 内核会将等待进程的退出状态写入调用进程的 `status` 指向的内存区域, 并销毁等待进程的内核栈及其 PCB。

代码片段 6.8 进程等待的实现 (第二版): 从等待进程的 PCB 中获取退出状态, 并销毁其 PCB

```

1 void process_waitpid_v2(int id, int *status)
2 {
3     while (TRUE) {
4         bool not_exist = TRUE;
5         // 扫描内核维护的进程列表, 寻找对应进程
6         for (struct process *proc : all_processes) {
7             if (proc->pid == id) {
8                 // 标记已找到的对应进程, 并检查其是否已经退出
9                 not_exist = FALSE;
10                if (proc->is_exit) {
11                    // 若发现该进程已经退出, 记录其退出状态
12                    *status = proc->exit_status;
13                    // 销毁该进程的内核栈
14                    destroy_kern_stack(proc->stack);
15                    // 回收进程的 PCB 并返回
16                    destroy_process(proc);
17                    return;
18                } else {
19                    // 如果没有退出, 则调度下个进程执行
20                    schedule();
21                }

```

```
22     }
23 }
24 // 如果列表中不存在该进程，则直接返回
25 if (not_exist)
26     return;
27 }
28 }
```

6.1.6 进程等待的范围与父子进程关系

前文介绍的进程等待系统调用存在一个问题：进程可以调用 `process_waitpid` 等待任何进程退出，并获取其退出状态。举个例子，假设小明和小红都在使用 `shell` 运行程序，小明就可以通过调用 `process_waitpid` 获取小红创建进程的退出状态，但小红可能并不想将退出状态泄露给小明。为了满足这一需求，我们需要对 `process_waitpid` 可以等待的进程进行限制。

那么，进程之间存在什么样的关系才能调用 `process_waitpid` 呢？一种通常的实现方式是：如果一个进程创建了另一个进程，那么创建者可以等待被创建者。这是因为创建关系通常暗含管理与被管理的关系：在创建新进程后，创建者一般需要管理该进程，就像上面例子中的 `shell` 进程管理 `grep` 进程。为了记录这种创建关系，我们将创建者称为父进程，将被创建者称为子进程，并利用这种创建关系限制 `process_waitpid` 的行为。

代码片段 6.9 首先展示了引入创建关系后 `PCB` 结构的变化。为了维护进程间的创建关系，每个 `PCB` 都加入了包含其所有子进程的列表 `children`，并在进程创建时进行相应的维护（已省略）。在此基础上，`process_waitpid` 的第三版实现（代码片段 6.10）只扫描自己的子进程列表，如果找不到对应的 `pid` 就会直接退出，从而限制了可等待的进程范围。

代码片段 6.9 PCB 结构（第四版）：加入子进程列表 `children`

```
1 struct process_v4 {
2     // 处理器上下文
3     struct context *ctx;
4     // 虚拟地址空间（包含页表基地址）
5     struct vmSPACE *vmSPACE;
6     // 内核栈
7     void *stack;
8     // 进程标识符
9     int pid;
10    // 退出状态
11    int exit_status;
12    // 标记是否退出
13    bool is_exit;
14    // 子进程列表
```



```

15  pcb_list *children;
16 };

```

代码片段 6.10 进程等待的伪代码实现 (第三版): 只等待自己的子进程

```

1 void process_waitpid_v3(int id, int *status)
2 {
3     // 如果没有子进程, 直接返回
4     if (!curr_proc->children)
5         return;
6     while (TRUE) {
7         bool not_exist = TRUE;
8         // 扫描子进程列表, 寻找对应进程
9         for (struct process *proc : curr_proc->children) {
10             if (proc->pid == id) {
11                 // 标记已找到的对应进程, 并检查其是否已经退出
12                 not_exist = FALSE;
13                 if (proc->is_exit) {
14                     // 若发现该进程已经退出, 记录其退出状态
15                     *status = proc->exit_status;
16                     // 销毁该进程的内核栈
17                     destroy_kern_stack(proc->stack);
18                     // 回收进程的 PCB 并返回
19                     destroy_process(proc);
20                     return;
21                 } else {
22                     // 如果没有退出, 则调度下个进程执行
23                     schedule();
24                 }
25             }
26         }
27         // 如果子进程列表中不存在该进程, 则立即退出
28         if (not_exist)
29             return;
30     }
31 }

```

小知识：进程资源的隐式回收

读者可能会有疑问, 由于进程等待的调用并不是强制的, 如果父进程不调用 `process_waitpid`, 那么子进程 PCB 是不是无法得到回收, 进而导致可用的内存资源越来越少?

针对这个问题, 通常的解决方法是交给一个专门的进程进行回收。例如在 Linux 中, 当父进程退出时, 它的所有子进程会被操作系统创建的第一个进程 (init 进程) “继承”, 即变为 init 进程的子进程。当这些子进程退出时, 内核会通知 init 进程完成对于这些进程的资源回收。

练习

- 6.1 小明需要实现一个包含两个进程的程序，其中第一个进程接收用户输入的数字，而第二个进程则将该数字输出到屏幕上。如果使用本节学习的进程相关系统调用，该如何实现该程序（文字描述即可）？

6.1.7 进程睡眠的实现

`process_waitpid` 实现了一种使进程在内核中陷入等待的机制。但进程除了等待特定事件（如其他进程退出）以外，有时也需要等待一定时间之后再继续执行。举个例子，小红编写了一个日程提醒程序，在日程开始前十分钟会自动通知。显然，该程序对应的进程只有等到具体时间才需要运行（发送通知），其他时候无须运行。为了实现该功能，就可以引入另一种等待固定时间的系统调用——睡眠。

代码片段 6.11 展示了进程睡眠相关的系统调用——`process_sleep` 的实现。该系统调用相对比较简单：内核首先获取当前时间作为睡眠起始时间，判断等待时间是否超过了参数中传入的时间长度（`seconds`，以秒为单位）。若时间未到，则调用与 `process_waitpid` 中相同的 `schedule` 函数在内核中等待。当等待的时间超过参数中指定的时间长度后，`process_sleep` 就从 `while` 循环中退出，返回用户态执行。

代码片段 6.11 进程睡眠的伪代码实现

```
1 void process_sleep(int seconds)
2 {
3     // 获取当前时间作为睡眠起始时间
4     struct *date start_time = get_time();
5     while (TRUE) {
6         struct *date cur_time = get_time();
7         if (time_diff(cur_time, start_time) < seconds) {
8             // 如果时间未到，则调度下个进程执行
9             schedule();
10        } else {
11            // 时间已到，直接返回
12            return;
13        }
14    }
15 }
```

6.1.8 进程执行状态及其管理

在学习进程相关系统调用的过程中，可以发现进程的执行状态并不是一成不变的，它们有时在 CPU 上运行，有时在内核中等待，有时则在退出后变为“僵尸”。系统调用通常是进程执行状态变化的关键节点。图 6.3 展示了进程可处于的五种执行状态，并标

出了引起进程执行状态变化的典型事件。



图 6.3 进程的五状态模型

- 新生（New）状态。当 `process_create` 被调用后，内核通过分配 PCB 创建了一个全新的进程。此时，该进程的初始化还没完成，不能执行程序。
- 就绪（Ready）状态。当 `process_create` 执行完成后，进程已经准备好开始执行程序了。但是，计算机中的进程数量通常较多，刚创建完成的进程可能无法立即开始执行，它们会在内核中等待。
- 运行（Running）状态。当操作系统的调度器（详见第 7 章）选择某个进程作为下一个执行的进程时，其状态就切换至运行状态，而前一个处于运行状态的进程由于暂停执行，又切换回就绪状态。当进程在 `process_waitpid` 或 `process_sleep` 里面调用 `schedule` 函数时，它就会切换到就绪状态，而操作系统会选出下一个进程并使其处于运行状态。
- 僵尸（Zombie）状态。前面已经提到，在子进程退出后，由于父进程可能会调用 `process_waitpid` 获取其退出状态，因此子进程的 PCB 并不会马上销毁。此时，虽然子进程退出，但它所占据的资源（主要是 PCB）没有完全被操作系统回收，该执行状态称为僵尸状态。
- 终止（Terminated）状态。当一个进程退出且其占据的资源全部被操作系统回收时，它就进入了终止状态，这是进程生命周期的终结。

维护进程的执行状态对于进程管理非常重要。从之前的例子可以看出，判断一个进程是否为僵尸状态可以帮助 `process_waitpid` 回收进程资源。另一个例子是调度操作，即选出下一个执行的进程：由于只有处于就绪状态的进程才能被调度，操作系统需要准确识别哪些进程处于这一执行状态。代码片段 6.12 和 6.13 展示了 `schedule` 函数的一种实现方法：该函数调用 `pick_next` 选取下一个执行的进程（为了简便，这里假设进程列表中始终有进程可以被调度）。由于操作系统在 PCB 中维护了进程所处的执行状态 `exec_status`，因此 `pick_next` 在遍历进程列表的过程中只选取那些处于就绪状态的进程作为下一个执行的进程。在 `pick_next` 选出下一个进程之后，`schedule` 将 `curr_proc` 赋值为该进程，并将其执行状态置为运行状态，从而完成调度工作。另外，由于 `exec_status` 已经包含僵尸和退出状态，因此这个版本的 PCB 去掉了用于维护是否退出的变量 `is_exit`。

代码片段 6.12 PCB 结构 (第五版): 加入进程执行状态, 去掉 is_exit

```
1 enum exec_status {NEW, READY, RUNNING, ZOMBIE, TERMINATED};
2
3
4 struct process_v5 {
5     // 处理器上下文
6     struct context *ctx;
7     // 虚拟地址空间 (包含页表基地址)
8     struct vmSPACE *vmSPACE;
9     // 内核栈
10    void *stack;
11    // 进程标识符
12    int pid;
13    // 退出状态
14    int exit_status;
15    // 子进程列表
16    pcb_list *children;
17    // 执行状态
18    enum exec_status exec_status;
19 };
```

代码片段 6.13 基于进程执行状态的调度选择的伪代码实现

```
1 struct process* pick_next(void)
2 {
3     // 遍历进程列表, 寻找下一个可调度 (处于就绪状态) 的进程
4     for (struct process *proc : all_processes) {
5         if (proc->exec_status == READY) {
6             // 将上一个正在运行的进程变为就绪, 然后返回
7             curr_proc->exec_status = READY;
8             return proc;
9         }
10    }
11 }
12 void schedule()
13 {
14     curr_proc = pick_next();
15     // 将选中的进程执行状态变为运行
16     curr_proc->exec_status = RUNNING;
17 }
```

图 6.3 仅展示了比较基本的五种执行状态, 为了方便内核进行进程管理, 还可以引入更多的状态类型。例如在进程睡眠的例子中, 如果设定的时间过长, 调用 `process_sleep` 的进程可能会被内核调度很多次, 但都因为时间未到而再次调用 `schedule`, 白白浪费 CPU 资源。为了解决这一问题, 内核可以引入阻塞 (Blocked) 状态, 对应需要在内核中等待、无法马上回到用户态执行的进程。同样是进程睡眠的例子, 内核可以将调用 `process_sleep` 的进程执行状态改为阻塞状态, 从而避免其被调度, 节约 CPU 资

源。而当进程睡眠的时间超过参数指定的长度后，内核会将进程“唤醒”，即将其执行状态改回就绪状态。阻塞和唤醒机制的内核支持将在第9章详细介绍。

练习

- 6.2 在6.1.4节 grep 与 shell 的例子中，grep 进程在其生命周期中的执行状态是如何变化的？其状态变化是否存在多种可能？

6.2 案例分析：ChCore 微内核的进程管理

本节主要知识点

- 什么是能力组？其在 ChCore 微内核中有什么作用？
- 微内核的进程管理接口如何实现？
- 微内核的进程管理与宏内核存在哪些不同？

前面的章节主要从宏内核视角出发，介绍了进程相关数据结构及管理接口的实现。在宏内核中，与进程相关的数据结构（如 PCB）均放在内核中，管理接口也以系统调用的形式暴露给用户，核心功能全部在内核中完成。但在微内核中，包括进程管理在内的操作系统功能被拆分并移入用户态，因此与宏内核存在较大不同。本节将以 ChCore 为例，分析微内核的进程管理机制是如何实现的。

6.2.1 进程管理器与分离式 PCB

ChCore 采用微内核设计，它将操作系统的功能进行解耦，并将很多功能以模块形式移到用户态。进程管理也不例外，ChCore 将这部分功能移到用户态，其对应模块称为进程管理器（Process Manager）。当用户调用与进程管理相关的调用（如进程创建和终止）时，实际上是调用进程管理器，并由进程管理器与内核交互，最终达到管理进程的目的。这也使得 PCB 的结构由宏内核的集中式向分离式方向发展。图 6.4 展示了由进程管理器和内核共同管理的分离式 PCB 结构。从图中可以看出，进程的 PCB 结构被分割成两部分，分别放在内核态和用户态。

PCB 内核态部分：cap_group

ChCore PCB 在内核态的部分称为 cap_group，是能力组（Capability group）的简称。由于程序运行过程中需要不同类型的资源（如 CPU 和内存等），为了便于对资源进行管理，ChCore 对内核资源进行了抽象，每种资源对应一种类型的对象（Object），而访问某个具体对象所需的“凭证”就是能力（Capability）。由于 ChCore 将进程作为资源